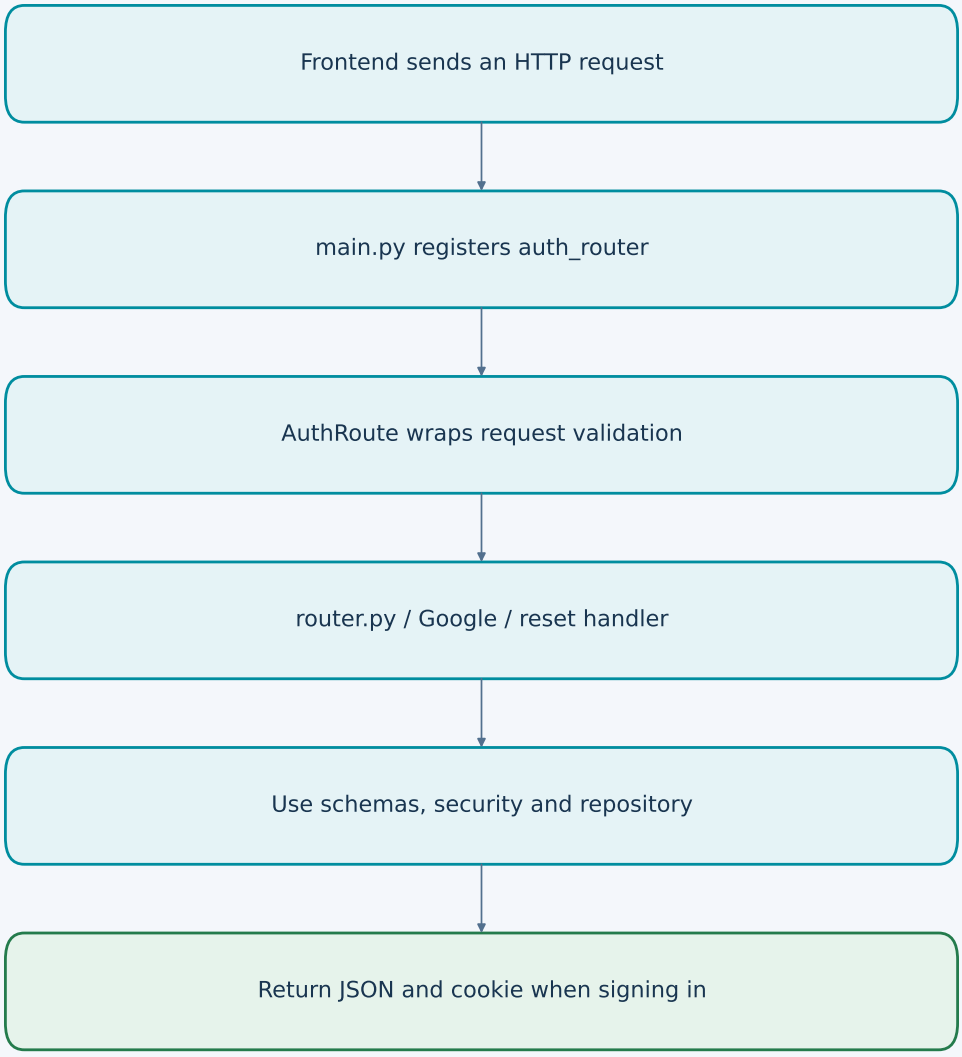
1. How the system is connected

Request path / ten authentication modules

The frontend sends JSON. FastAPI validates it, calls the matching handler, and uses PostgreSQL for account storage. Both login methods issue the same application token.



Who owns what?

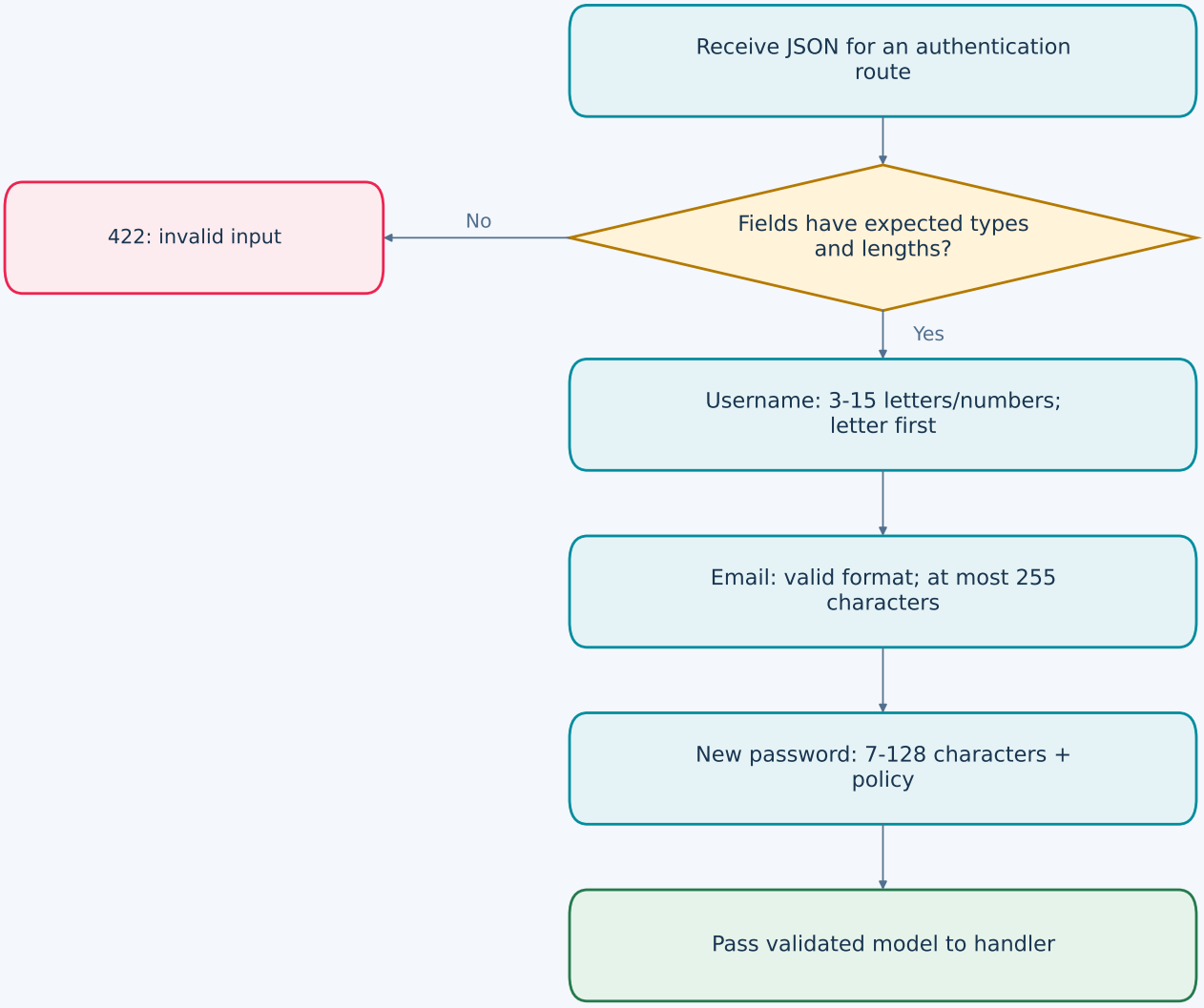
```
Schemas: input rules   |   Repository: persistent accounts
Security: hashes and JWTs   |   Current user: protected access
```

Authentication proves identity. Friendship and other features use that identity, then check which actions the user may perform. Read top to bottom; side arrows show alternatives.

2. Validate the submitted data

schemas.py / Pydantic request models

Validation runs before the endpoint body. Username rules are shared by normal registration and Google signup. Login accepts an existing password without applying new-password complexity.



New-password policy

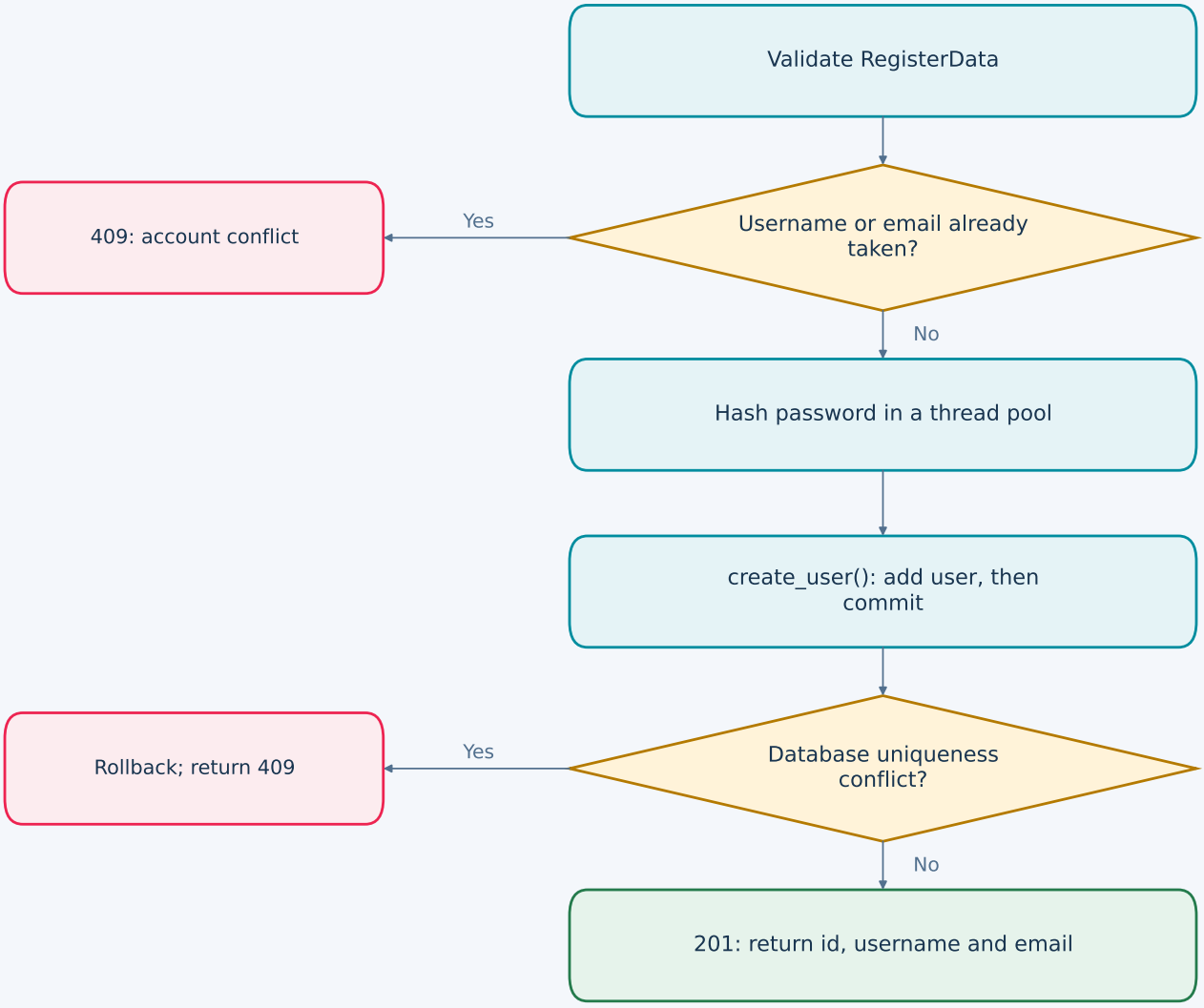
```
Uppercase + lowercase + number + symbol; no spaces
Login password: 1-128 characters; valid UTF-8 encoding
```

Username checks use isalpha()/isalnum(), so Unicode letters are allowed. Password uses SecretStr plus validate_password_encoding(). SecretStr masks representations; it does not encrypt.

3. Create a password account

POST /auth/register / router.register()

Registration checks availability, hashes the password, and saves a new user. It returns public account fields; the user must then log in separately.



Example

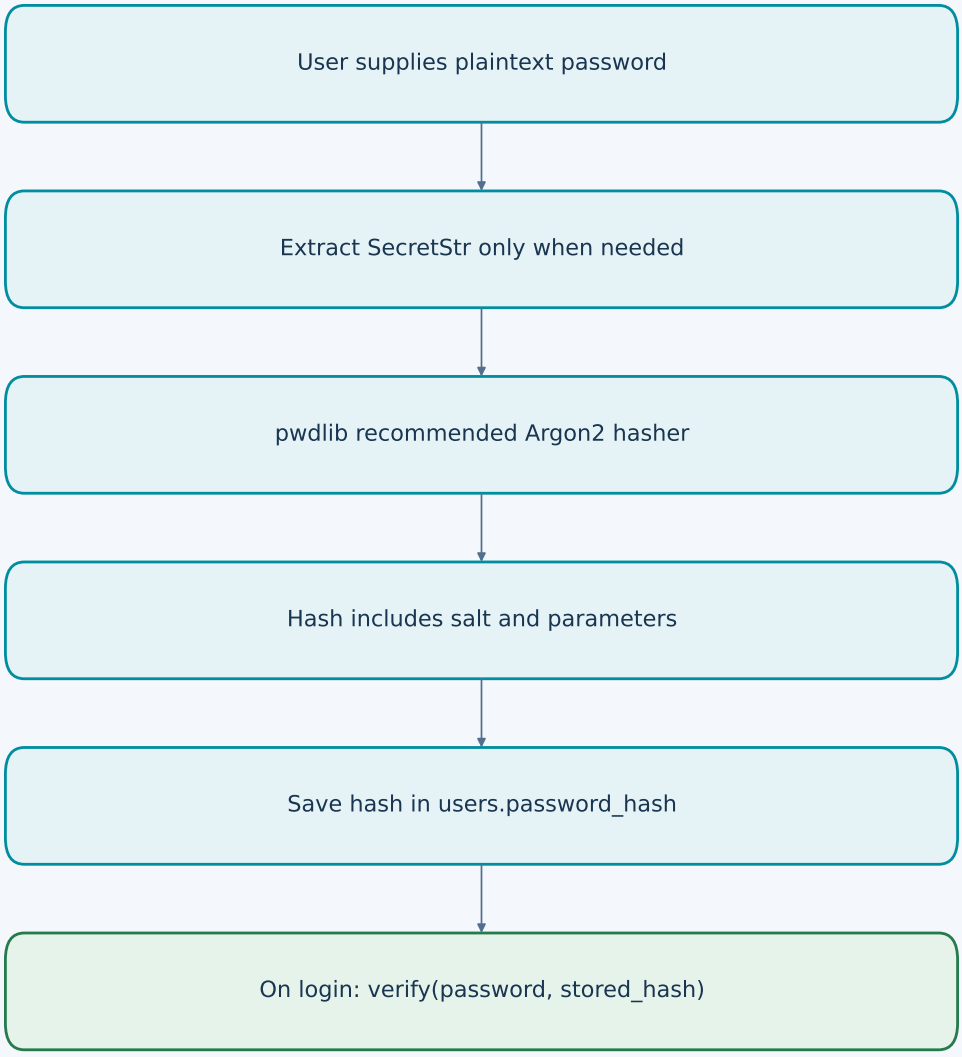
```
{"username":"PlayerOne","email":"player@example.com",
 "password":"MyNewPassword123!"}
```

registration_conflict() checks username before email. Unique database constraints also handle two requests racing for the same identity. Registration does not set a login cookie.

4. Understand password hashing

security.py / hash_password() and verify_password()

The backend stores a salted password hash. Login verifies the submitted password against that hash; it never decrypts a stored password.



Two different uses of hashing

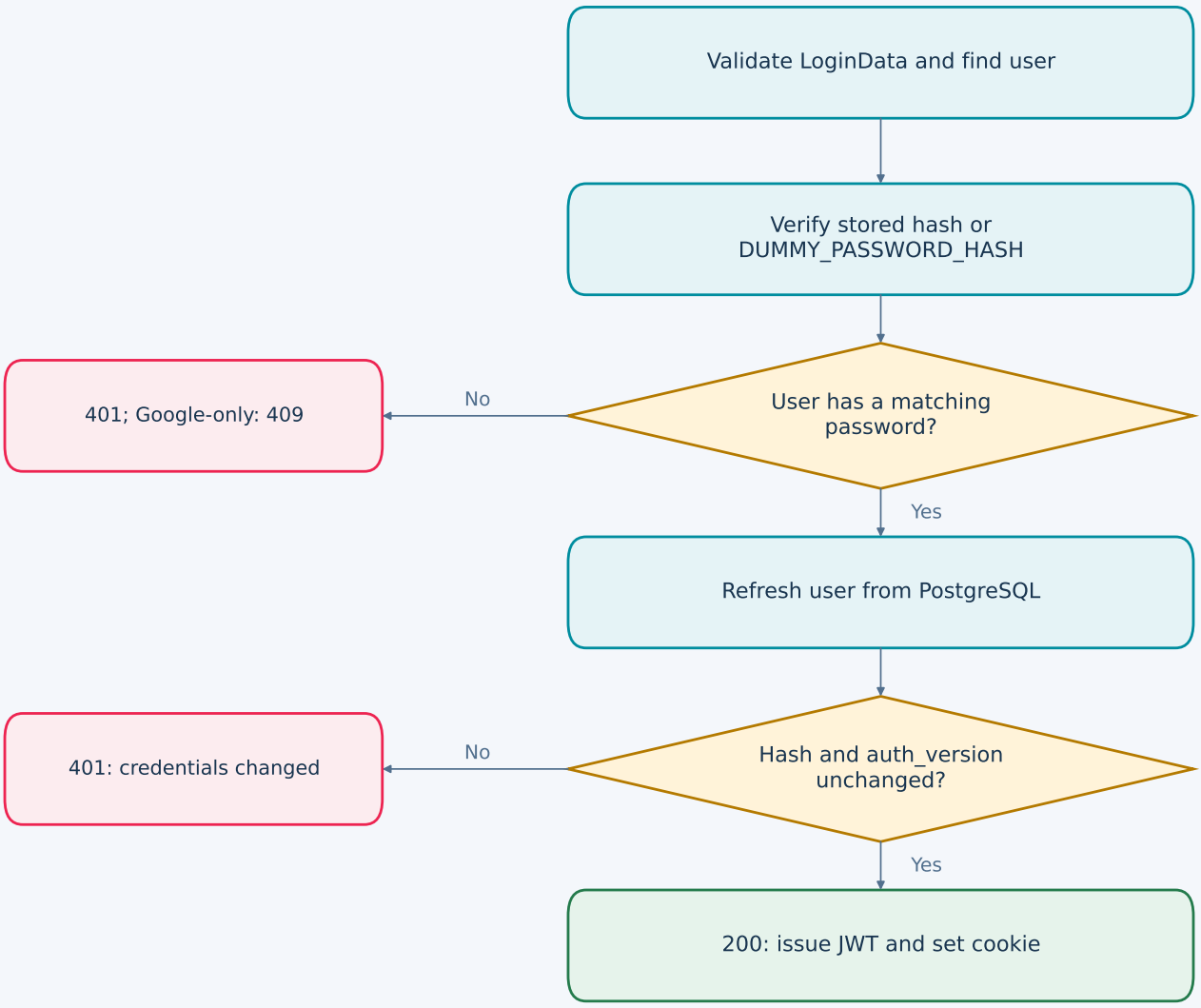
```
Argon2: user passwords (deliberately expensive)
SHA-256: random password-reset tokens (see page 13)
```

The salt is handled by the password hashing library; no separate salt column is needed. verify_password() returns False for an unknown hash format. Hashing/verification run off the event loop.

5. Log in with a password

POST /auth/login / authenticate_user()

Look up the account by normalized email, then verify the password. A dummy hash is used when no real password hash exists, reducing the difference in verification work.



Success response

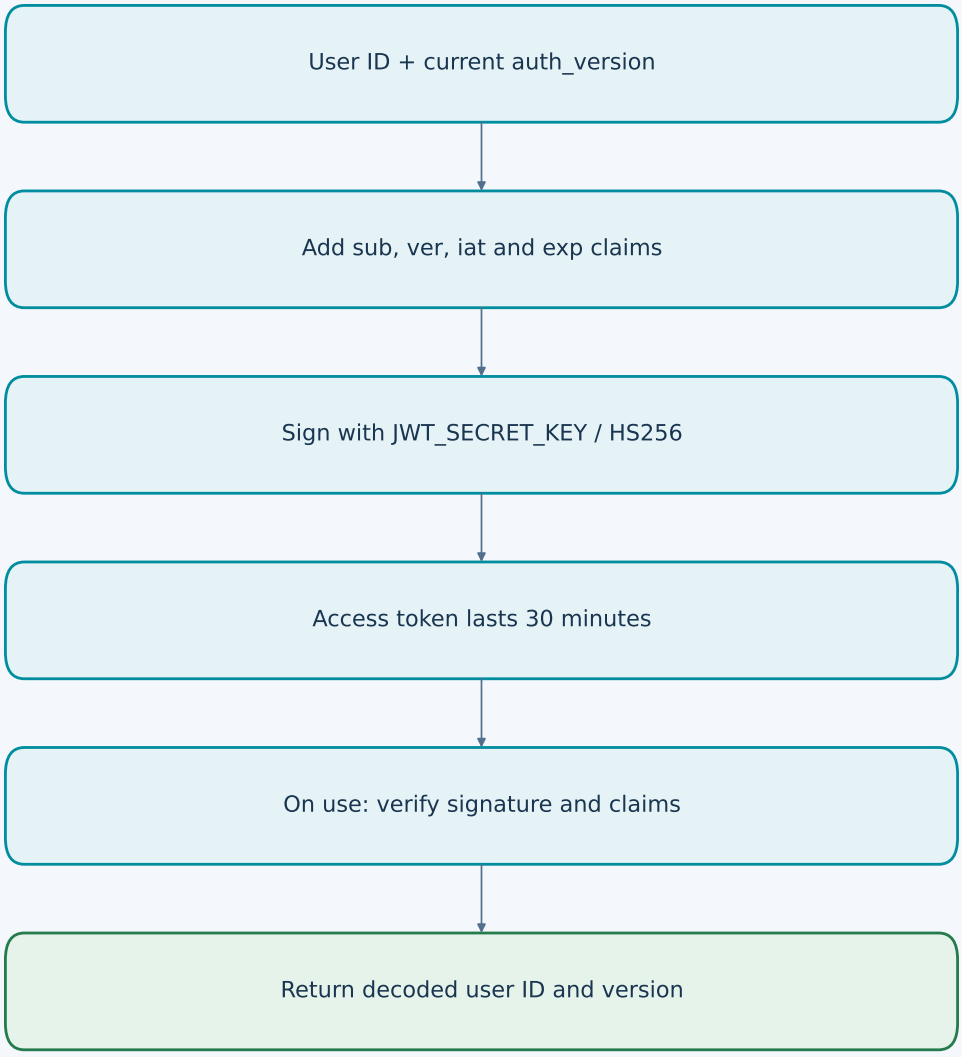
```
{"access_token":"APPLICATION_JWT","token_type":"bearer"}
```

Refreshing catches credential/version changes during password verification. Google-only accounts are told to use Google. The dummy hash does not make every response indistinguishable.

6. Create and verify the JWT

security.py / application access token

Both email and Google login create a signed application JWT. The payload is readable, so it contains identity and timing claims, never the password or signing secret.



Payload fields

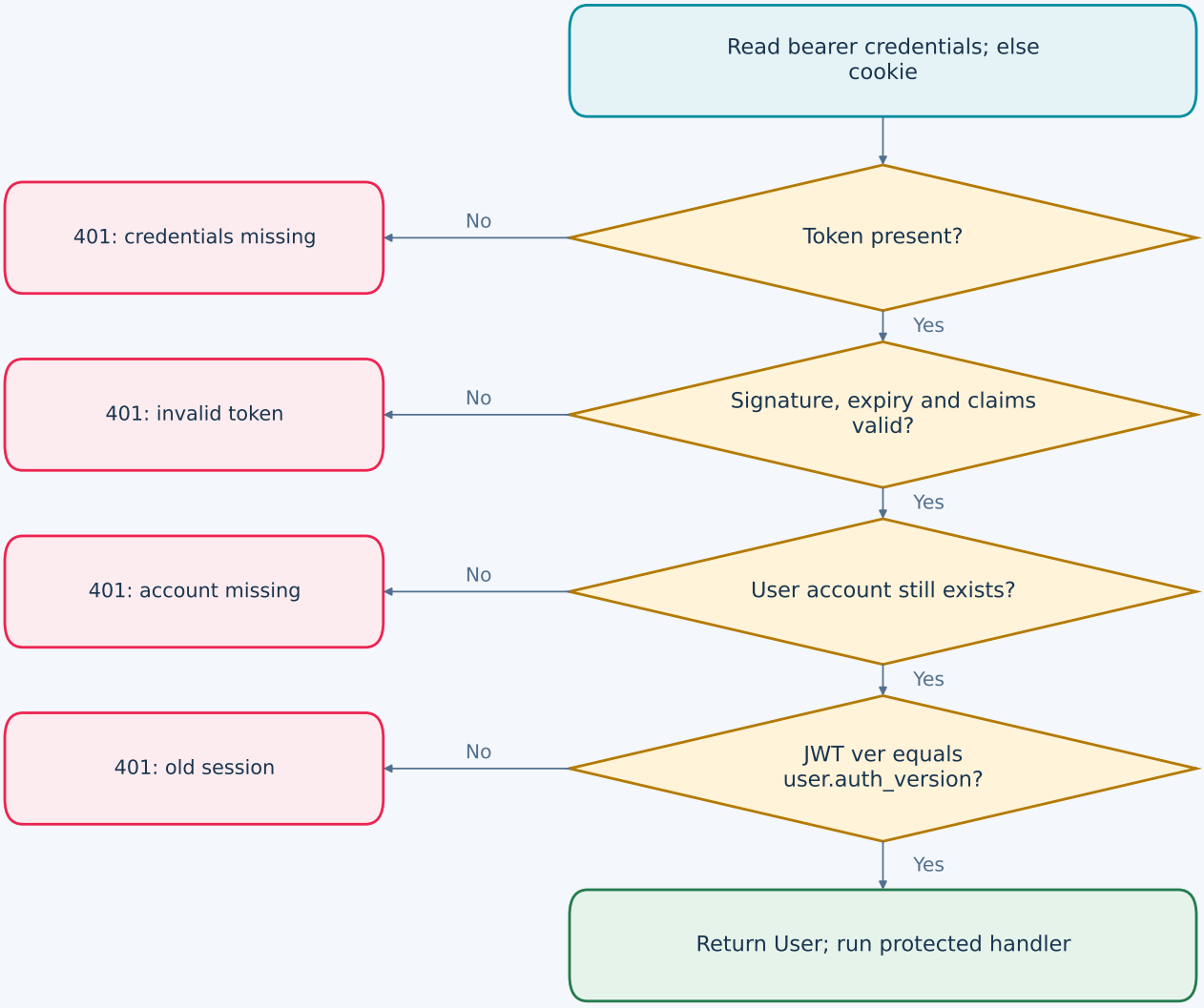
```
sub = "42"   ver = 0
iat = issued time   exp = issued time + 30 minutes
```

decode_access_token() requires all four claims, integer times, a positive decimal user ID within 1-2147483647, and a nonnegative integer version. PyJWT checks signature and time validity.

7. Authenticate a protected request

current_user.py / get_current_user()

The acting user comes from a verified token. FastAPI supplies the database User to the protected handler through Depends(get_current_user). A browser-supplied user ID is not proof.



Bearer or cookie

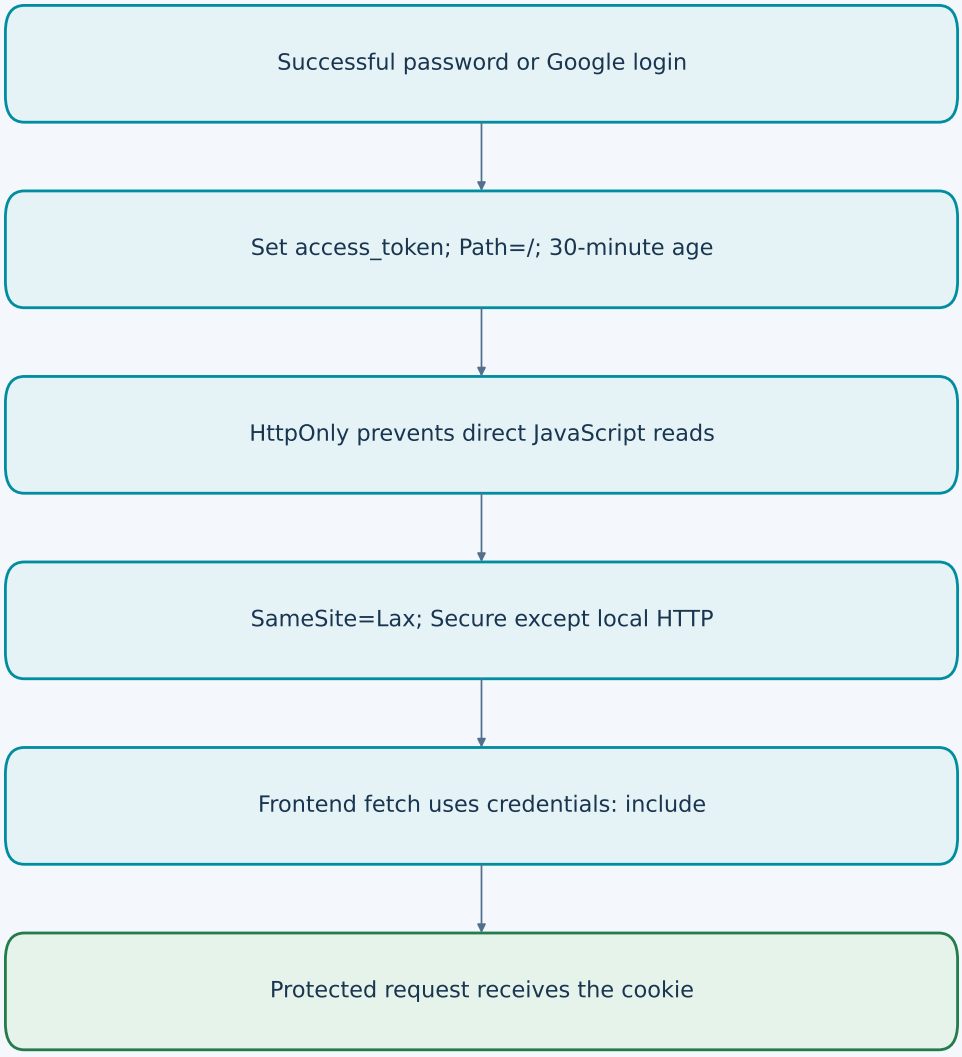
```
Authorization: Bearer APPLICATION_JWT
Cookie: access_token=APPLICATION_JWT
```

Bearer credentials take priority; an invalid bearer token does not fall back to the cookie. unauthorized() includes WWW-Authenticate: Bearer. Current HTTP dependencies contain no Origin check.

8. Keep the browser signed in

session_cookie.py / set_access_cookie()

A successful login writes the JWT into the access_token cookie and returns the same JWT in JSON. The browser uses the HttpOnly cookie for subsequent eligible requests.



Host matters

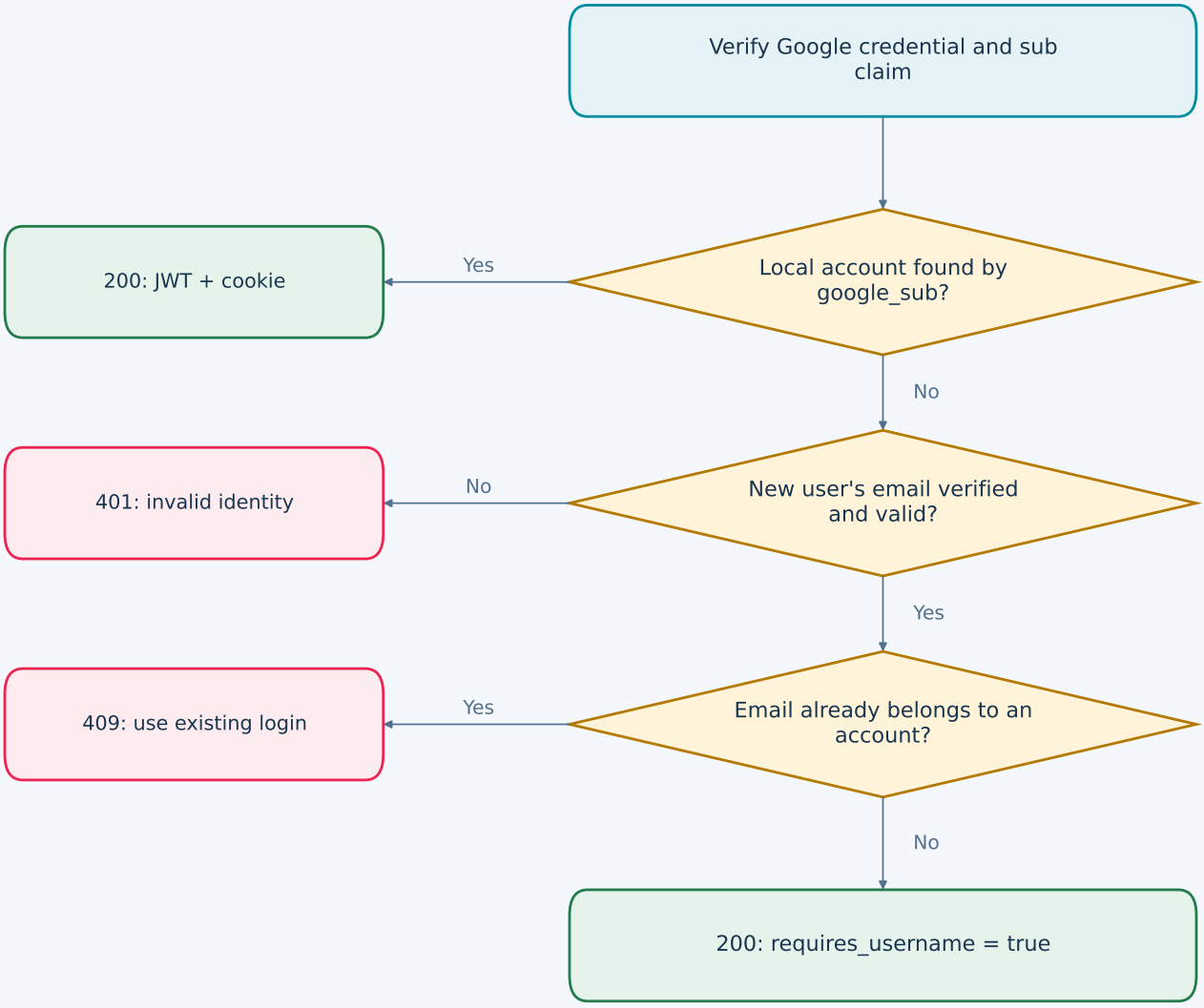
```
Use localhost consistently, or use 127.0.0.1 consistently.
Cookie Max-Age = 1800 seconds; JWT expiry = 30 minutes.
```

Secure is enabled for HTTPS or any non-loopback hostname. ALLOWED_BROWSER_ORIGINS is used by CORS in main.py. CORS and SameSite do not constitute a complete CSRF policy.

9. Start Google sign-in

POST /auth/google / google_login()

The browser supplies a Google ID token as credential. The backend verifies it for GOOGLE_CLIENT_ID, then uses Google's sub claim to find the local account.



First-time response

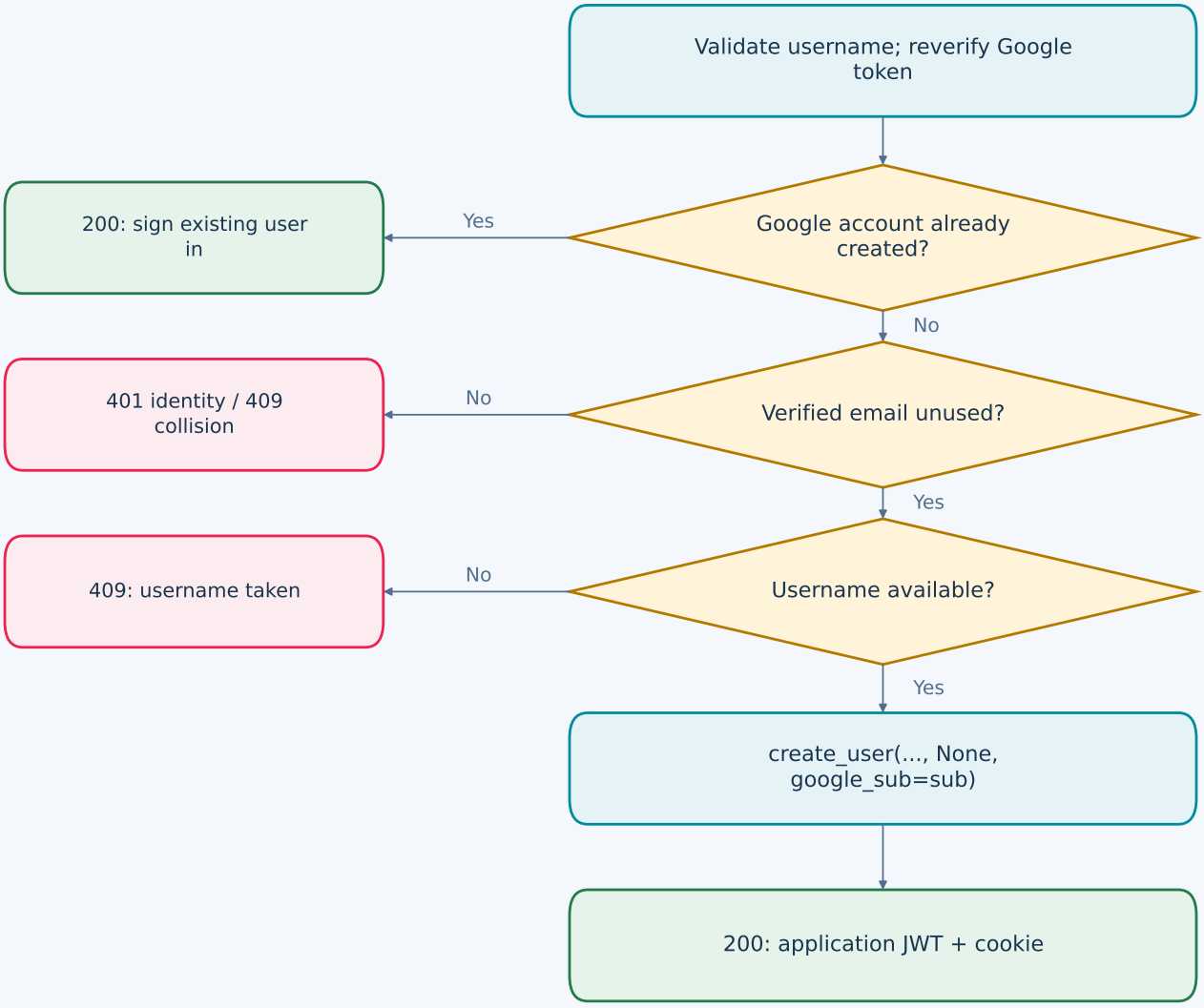
```
{"requires_username": true}
No placeholder user and no login cookie are created yet.
```

verify_google_token() maps missing configuration or transport failure to 503, and invalid credentials to 401. Existing accounts sign in with their saved username; there is no legacy-name check.

10. Choose the Google username

POST /auth/google/complete / complete_google_signup()

The second request contains the Google credential and chosen username. Verification runs again before account creation. GoogleCompleteData inherits the shared username rules.



Save to PostgreSQL

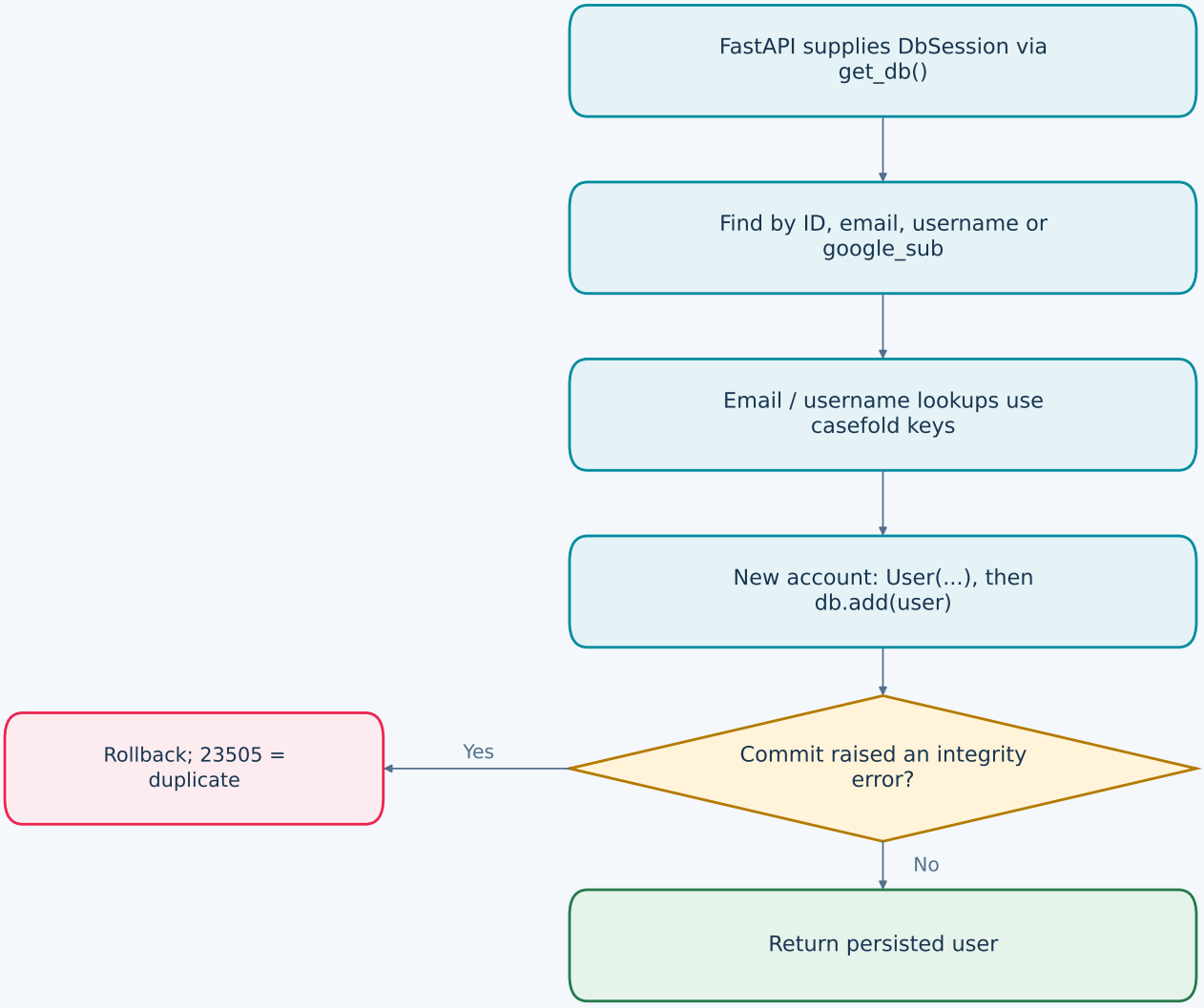
```
create_user() adds the user and commits the transaction.
password_hash = None; google_sub = verified Google identity
```

The existing-account branch permits retry after a saved signup response was lost; it does not rename that account. DuplicateUserError becomes 409. Email collisions never automatically link accounts.

11. Read and save account data

repository.py / PostgreSQL and SQLAlchemy

Both authentication methods share the users table. DbSession asks FastAPI for an async database session. Reads return a User or None; create_user() saves new accounts.



Database uniqueness

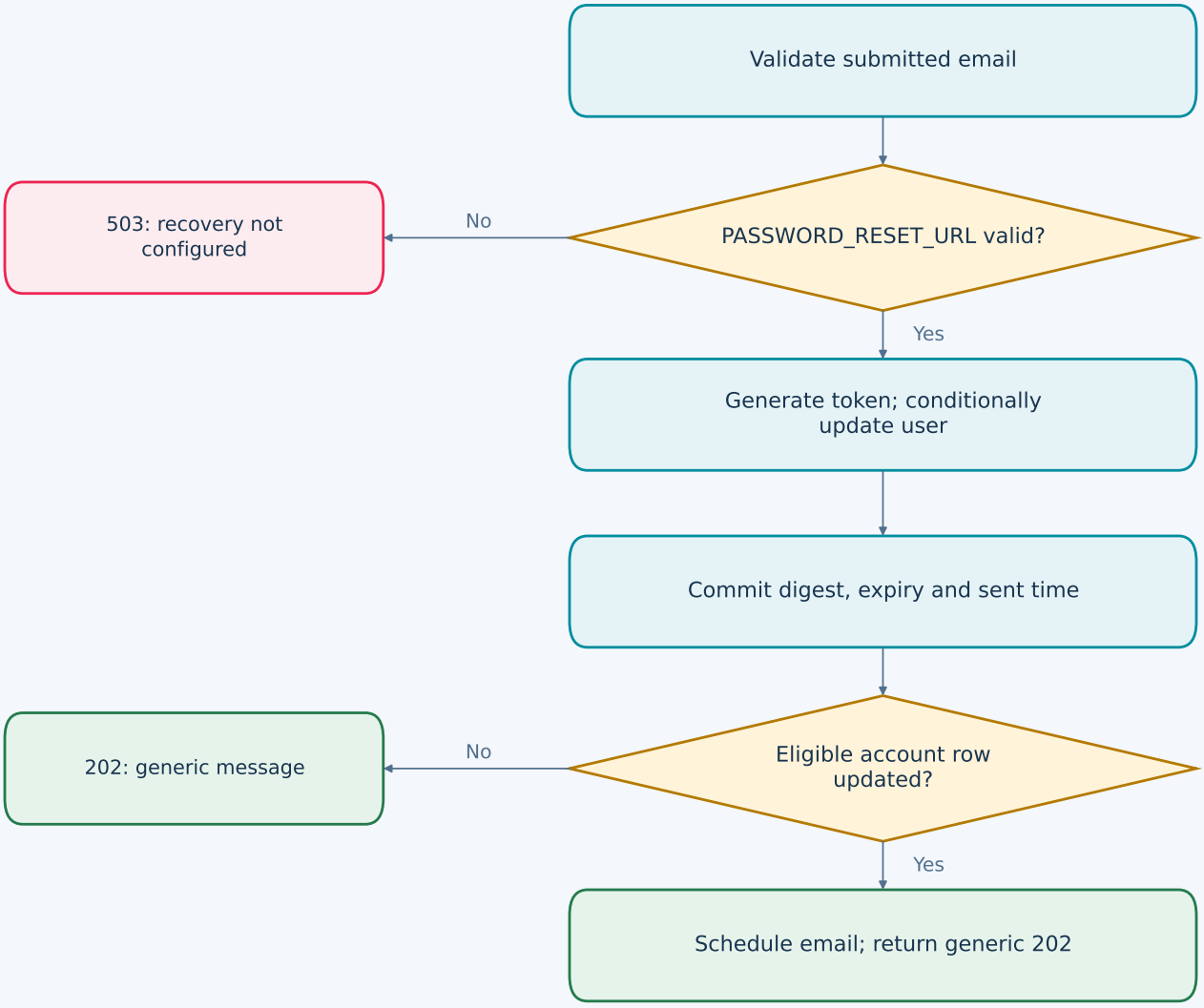
```
username_key / email_key / google_sub enforce uniqueness.
User.normalize_identity() fills normalized keys on ORM assignment.
```

find_user_by_id() accepts IDs 1-2147483647. create_user() translates unique violations to DuplicateUserError and reraises other failures. Request sessions close after use; writes need explicit commit.

12. Request a password reset

POST /auth/forgot-password / forgot_password()

Only accounts with a local password can receive reset mail. The response is deliberately generic for missing accounts, Google-only accounts and cooldown-limited requests.



Eligibility and response

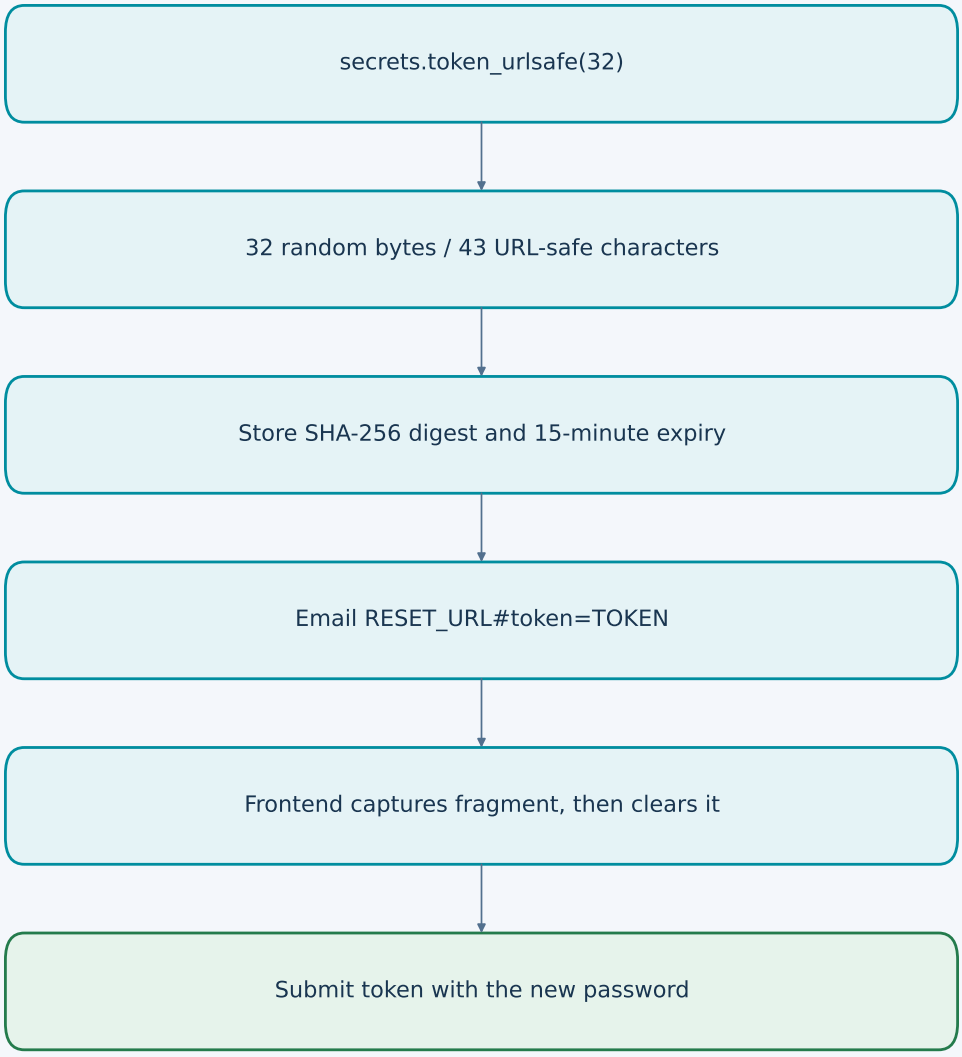
```
Has password_hash; last reset request at least 60 seconds ago.
"If an eligible account exists, a password reset email will be sent."
```

The URL must have an HTTP(S) scheme and hostname, with no fragment or embedded credentials. A new eligible request replaces the old digest. A 202 response does not confirm email delivery.

13. Understand the reset token

Random secret / stored digest / frontend fragment

A reset token is a temporary secret from your own backend. It is neither a Google credential nor the application JWT. The database stores its digest, not the usable token.



Storage and input rules

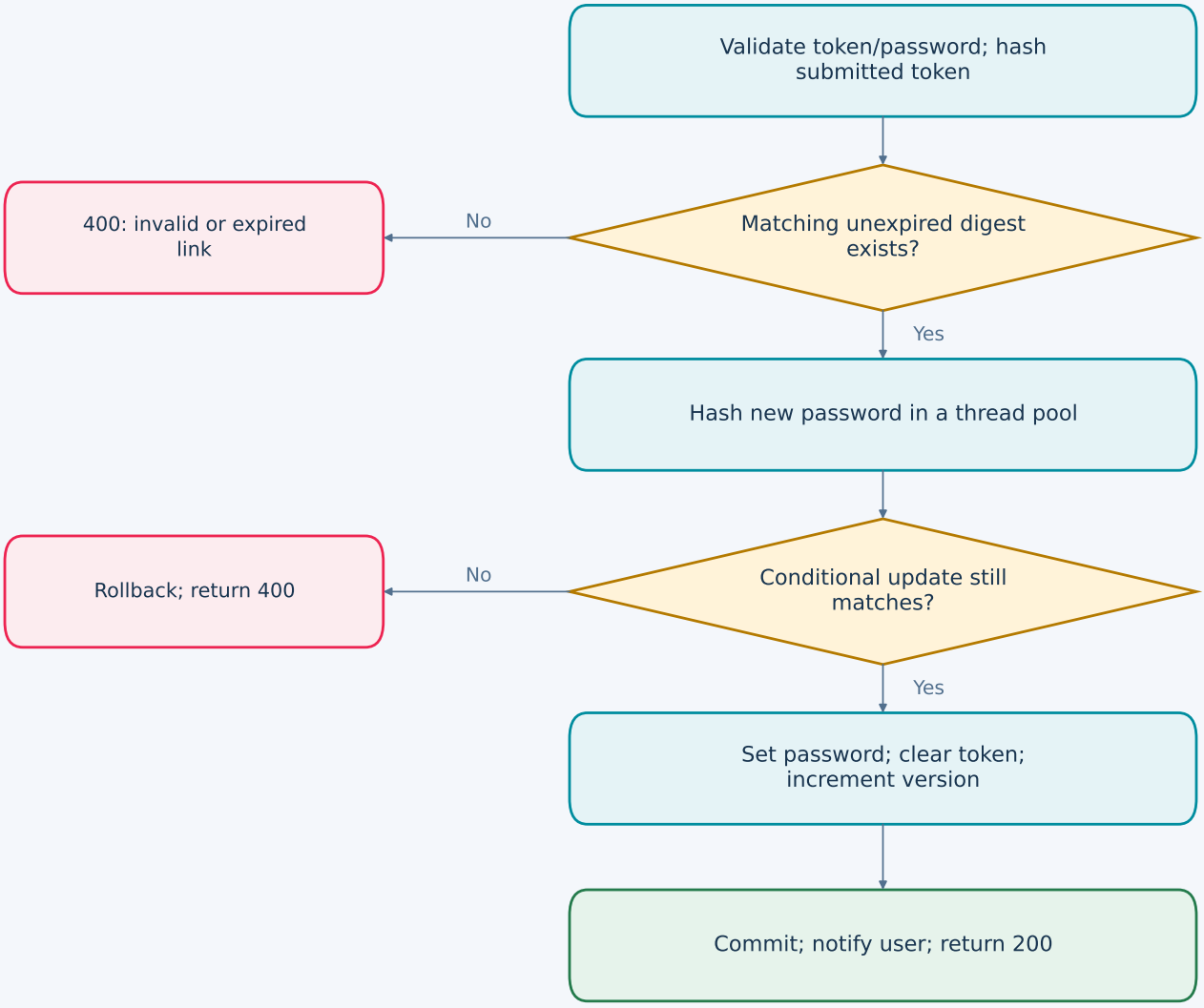
```
reset_digest: 64-character hexadecimal SHA-256 value
Input token: exactly 43 ASCII letters/numbers, '-' or '_'
```

ResetPasswordData.validate_token() enforces the token format. The frontend keeps the token in memory after clearing the URL. Reloading then requires reopening the email link. Use HTTPS in deployment.

14. Consume the reset link once

POST /auth/reset-password / reset_password()

Validate the new password using the registration policy. Look up the token digest, hash the new password, then recheck eligibility in the final database update.



One atomic update

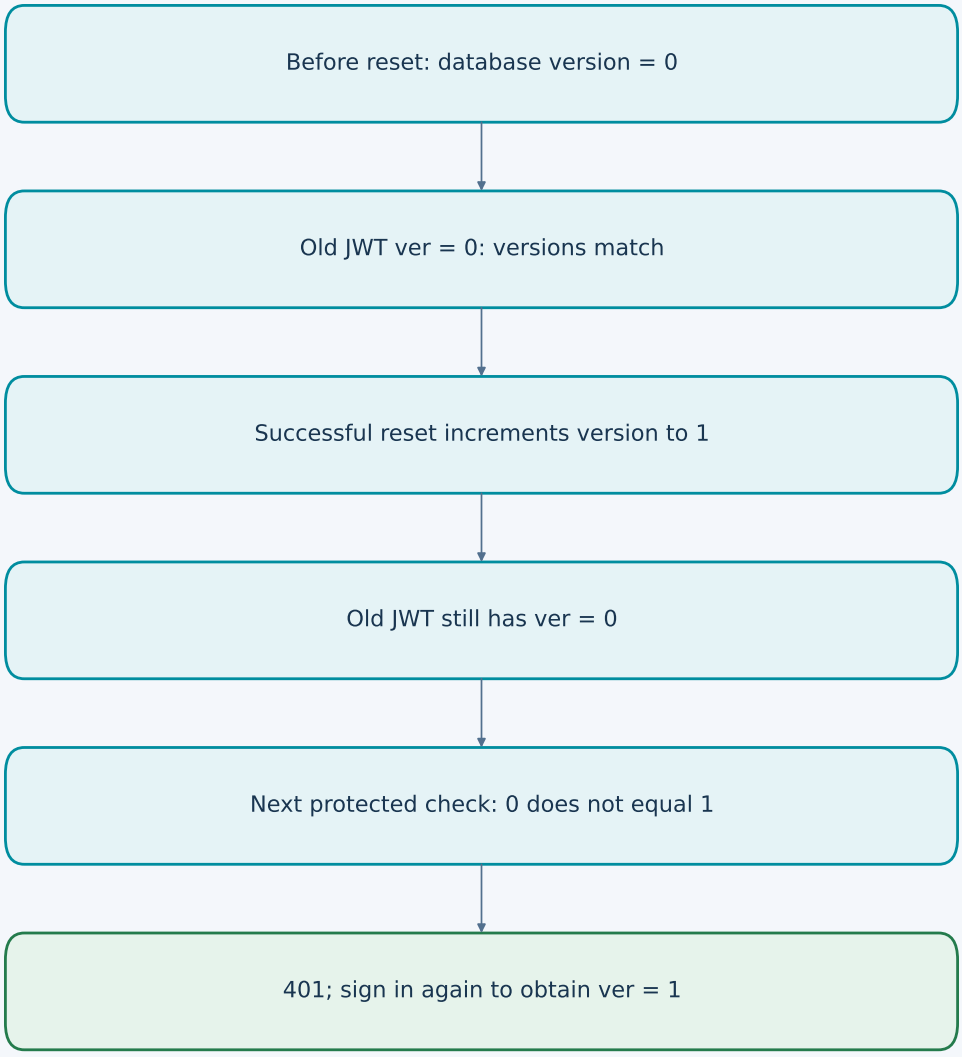
```
password_hash = new hash; reset_digest = NULL
reset_expires = NULL; auth_version = auth_version + 1
```

The second check prevents two competing consumers from succeeding and catches expiry while hashing. reset_sent_at stays set. Success schedules a confirmation email but does not issue a new login session.

15. Invalidate earlier sessions

auth_version / database check on protected access

Every JWT carries ver. The database stores auth_version. Increasing that value makes earlier tokens fail even when their signatures are valid and they have not expired.



What is revoked?

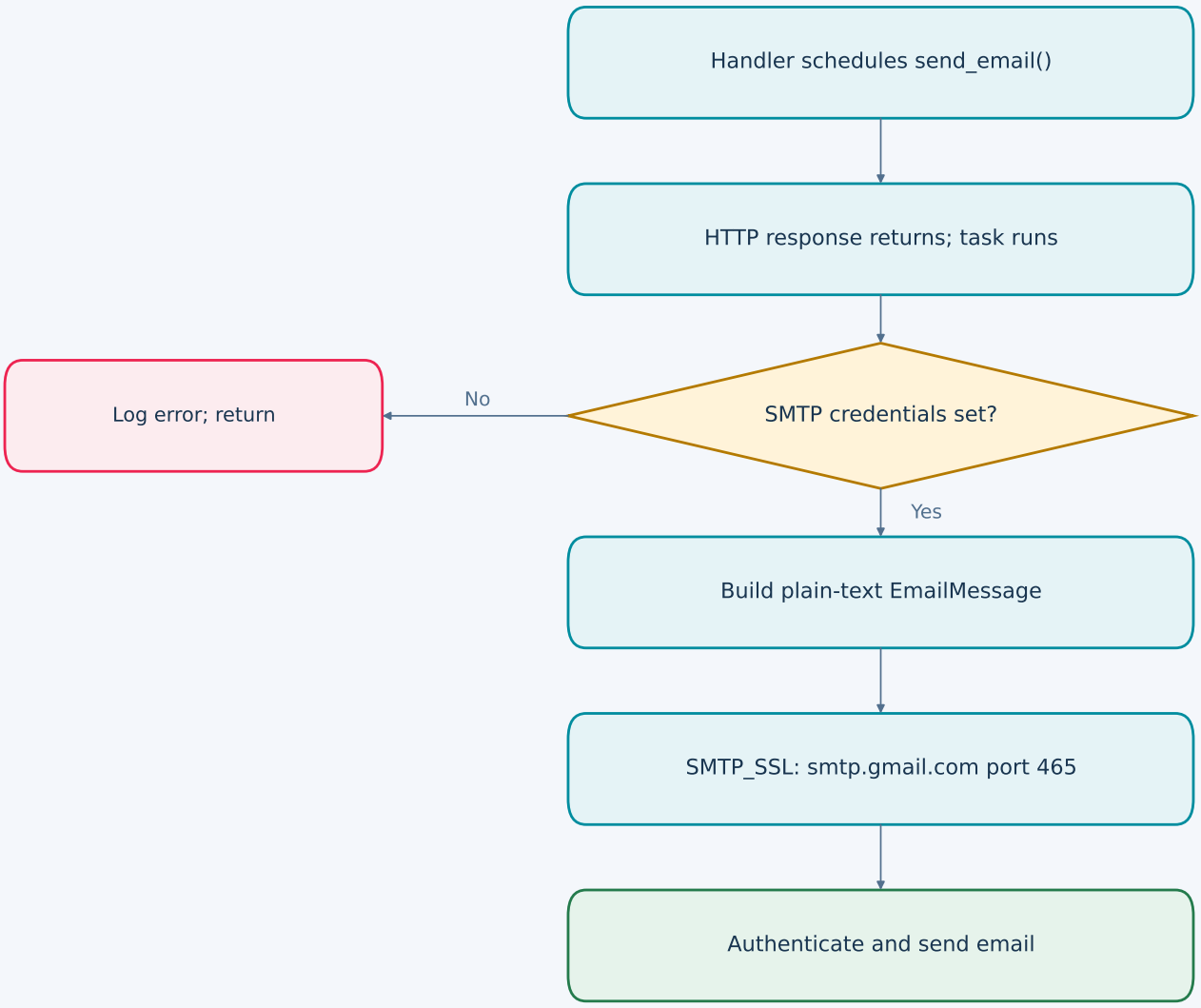
```
Old JWT on any device -> rejected at the next auth check.
New login after reset -> token contains the new version.
```

HTTP dependencies check the database for each protected request. WebSocket authentication runs when its dependency executes; this code does not itself disconnect already open sockets after a reset.

16. Send reset and confirmation mail

mailer.py / send_email() and BackgroundTasks

Reset handlers commit their database changes before delivery. FastAPI runs scheduled mail tasks after returning the HTTP response. Email failure does not undo a committed reset.



Delivery settings

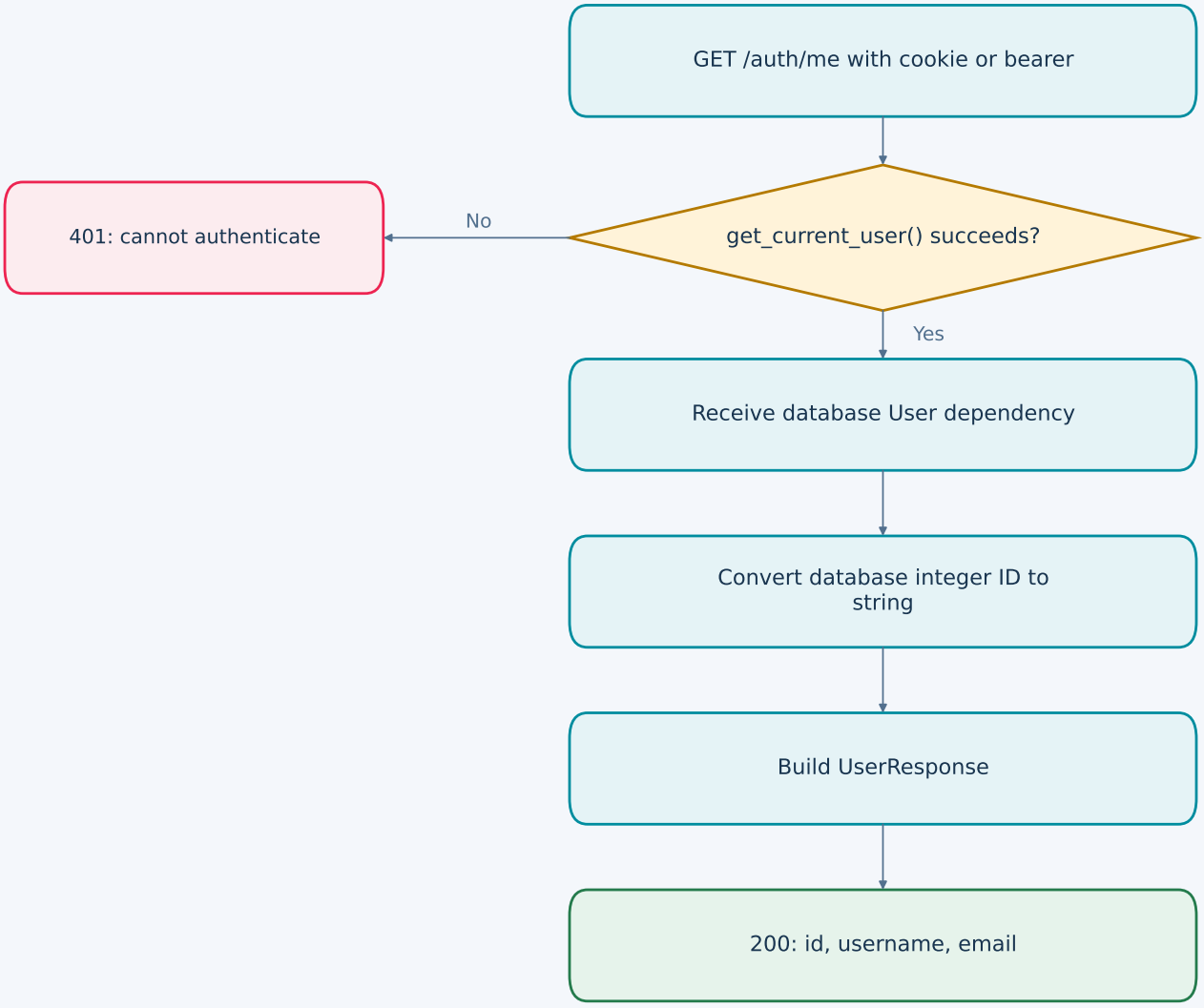
```
TLS context = ssl.create_default_context(); timeout = 10 seconds
Reset email contains link; confirmation email reports password change.
```

SMTP/network exceptions are logged without the token or credential value in the error message. There is no automatic retry or durable mail queue. Check logs and spam folders when a 202 produces no mail.

17. Read the current account

GET /auth/me / router.me()

The frontend uses this route to confirm the cookie session works. FastAPI authenticates the caller first, then the handler builds a small public identity response.



Example response

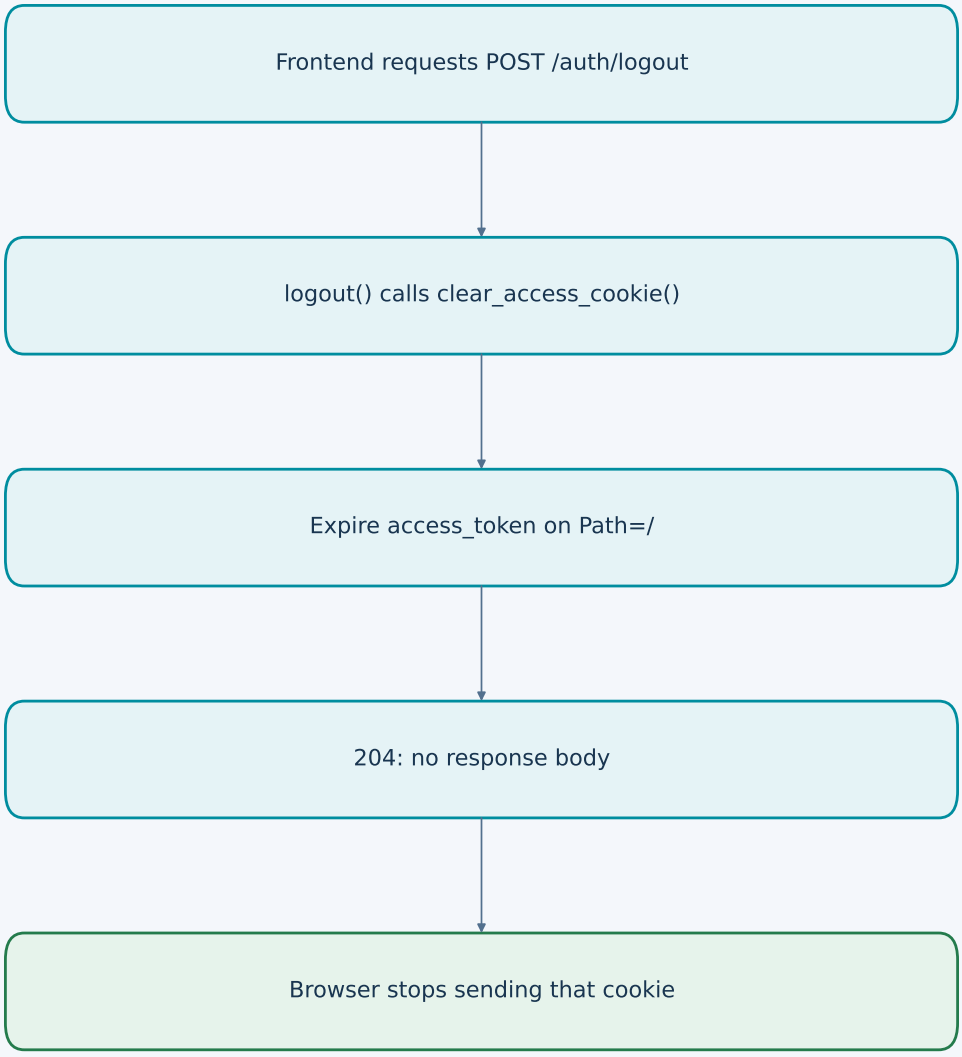
```
{"id":"42","username":"PlayerOne",
 "email":"player@example.com"}
```

No password hash, reset digest or Google credential is returned. /me does not issue a new JWT or extend expiry. Authentication identifies the caller; other endpoints still enforce their own permissions.

18. Log out of this browser

POST /auth/logout / clear_access_cookie()

Logout removes the browser's access_token cookie. This endpoint does not require a valid current session and does not change the user's database session version.



Logout versus password reset

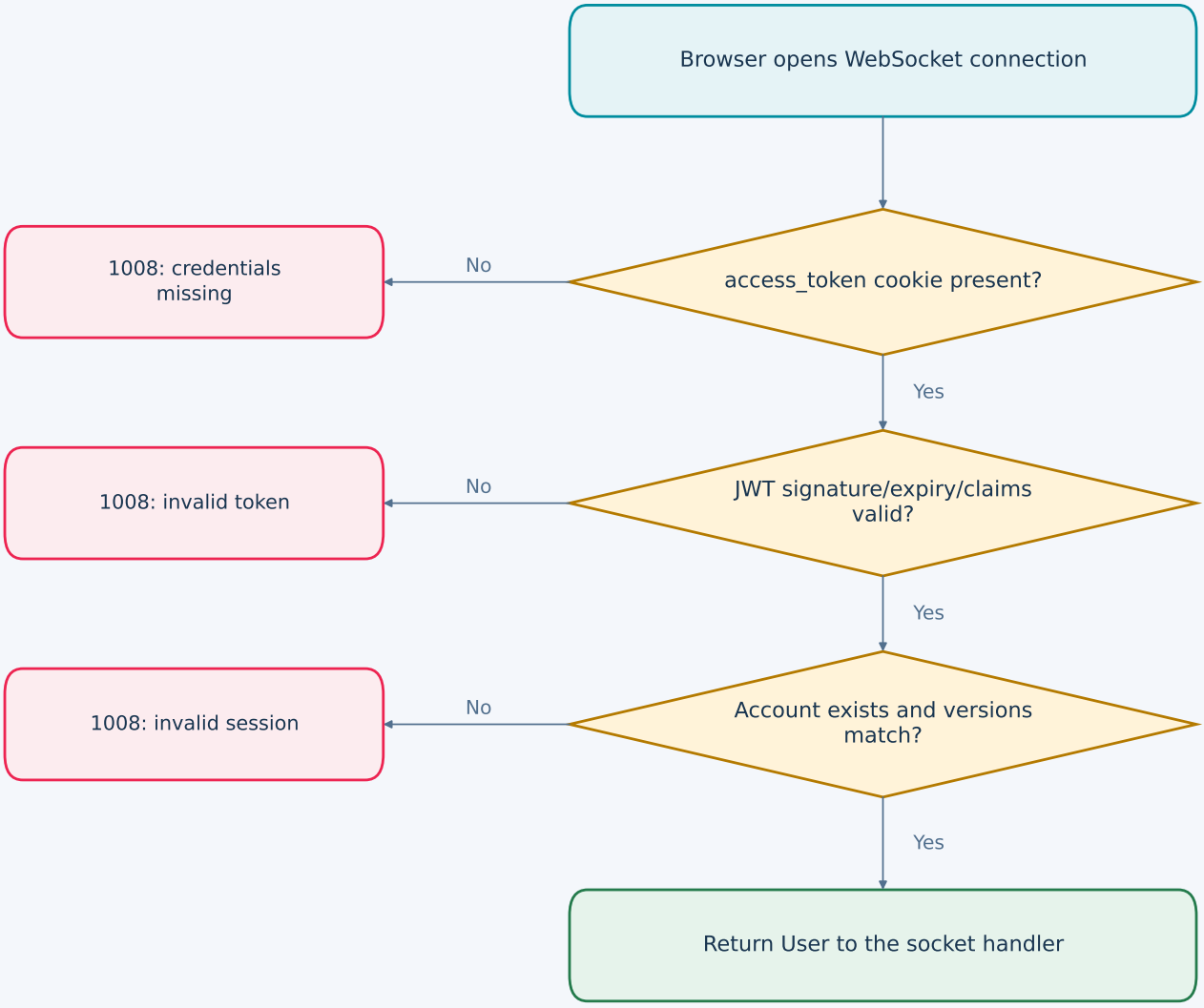
```
Logout: remove this browser's cookie.
Password reset: increase auth_version; reject old-version tokens.
```

A copied JWT remains usable until expiry or version invalidation. The frontend also tries to clear presence before logout; that users endpoint is outside authentication. There is no token blacklist here.

19. Authenticate a WebSocket

current_user.py / get_current_user_ws()

Room and invitation sockets reuse the same JWT checks as HTTP. This dependency reads the access_token cookie only; it does not accept an HTTP bearer credential argument.



Failure response

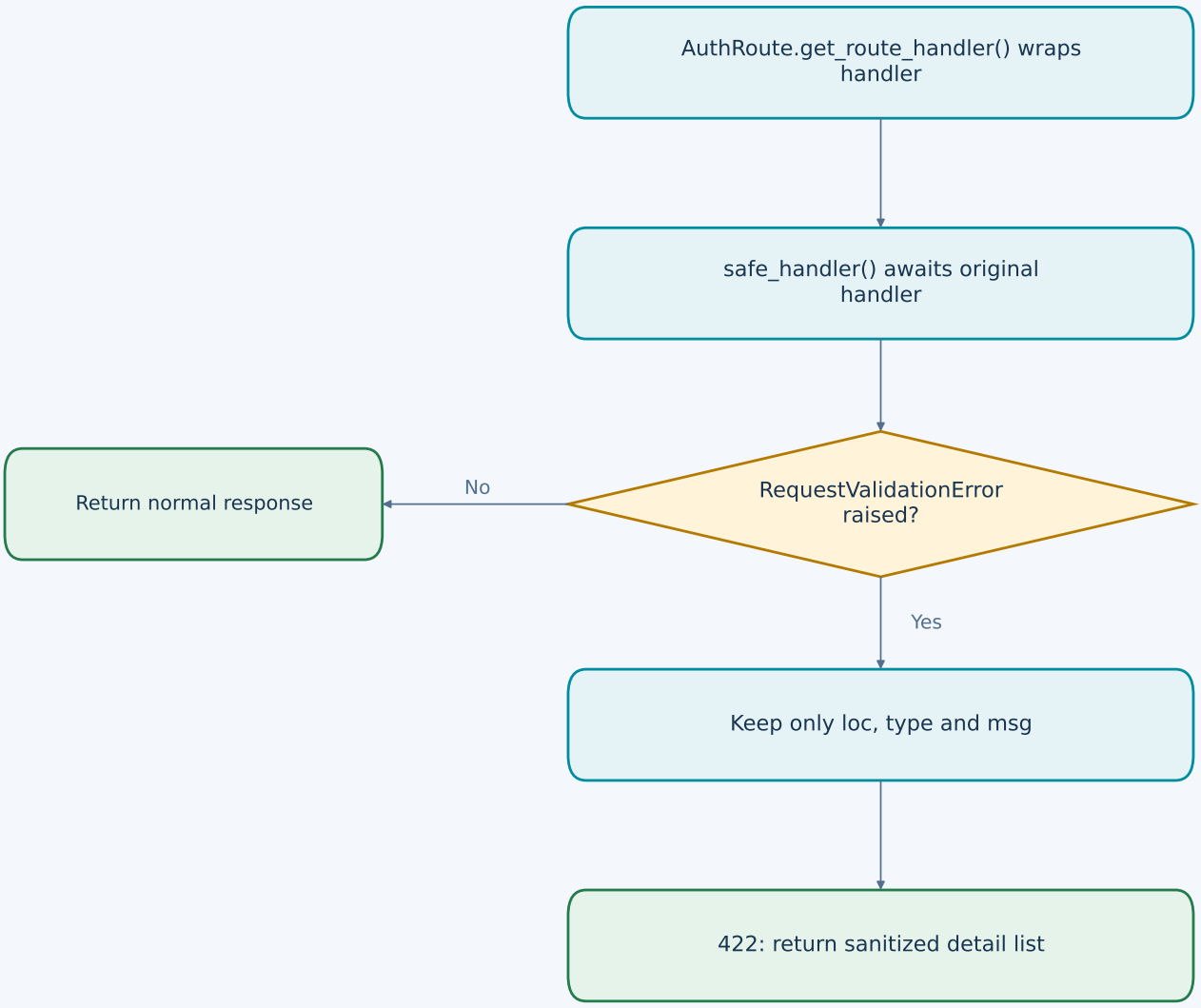
```
WebSocketException(code=1008,
 reason="Could not validate credentials")
```

1008 is a WebSocket policy violation, not an HTTP 401 response. This function authenticates at connection dependency time; it does not continuously recheck expiry/version for incoming messages.

20. Keep credentials out of errors

validation_route.py / AuthRoute

FastAPI validation errors can carry submitted input. AuthRoute wraps authentication handlers and builds a smaller error response containing only location, type and message.



Example error shape

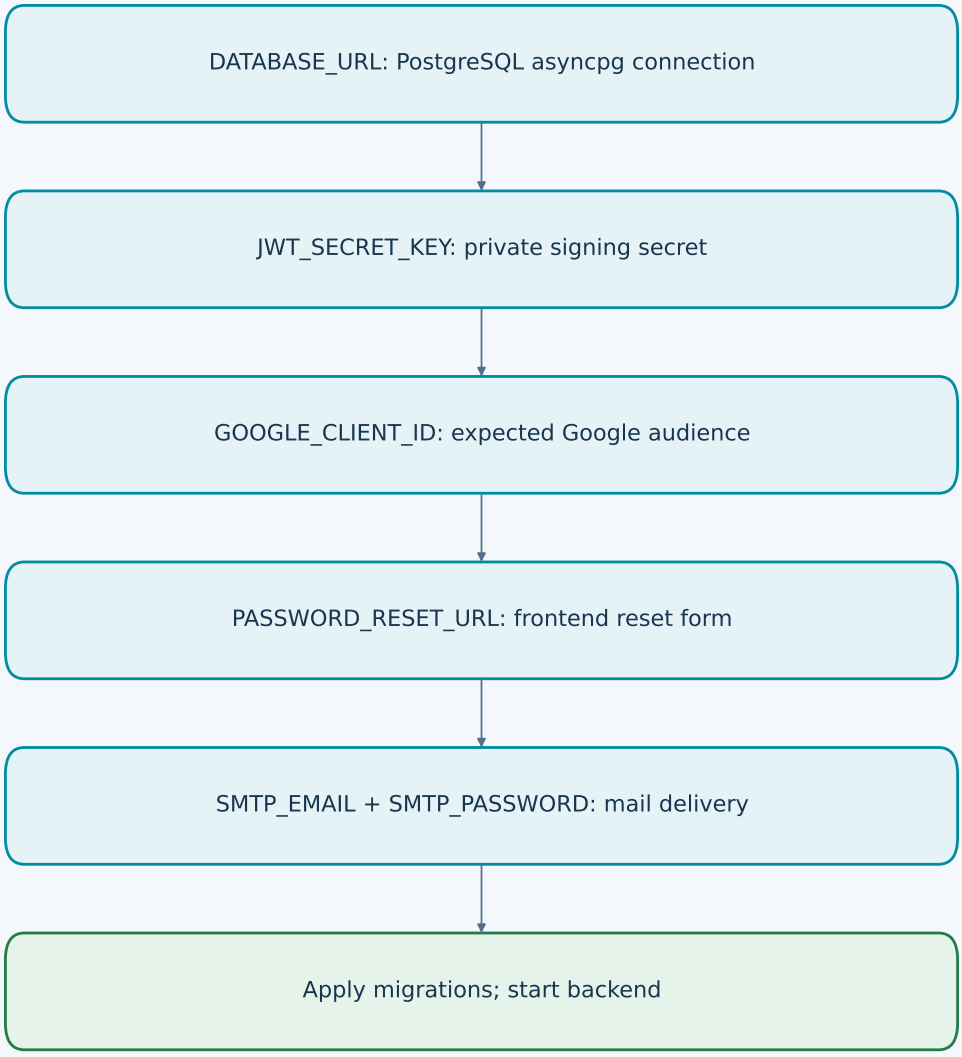
```
{"detail":[{"loc":["body","password"],
 "type":"value_error","msg":"Password policy error"}]}
```

Submitted input and validation context are omitted. Auth, Google and reset routers use this route class. It does not catch every exception or sanitize arbitrary logging elsewhere in the application.

21. Configure the authentication service

Process environment / deployment integration

These modules read os.environ. The launch environment must provide database and JWT configuration before imports. Optional features require their own settings when used.



Current Compose setup

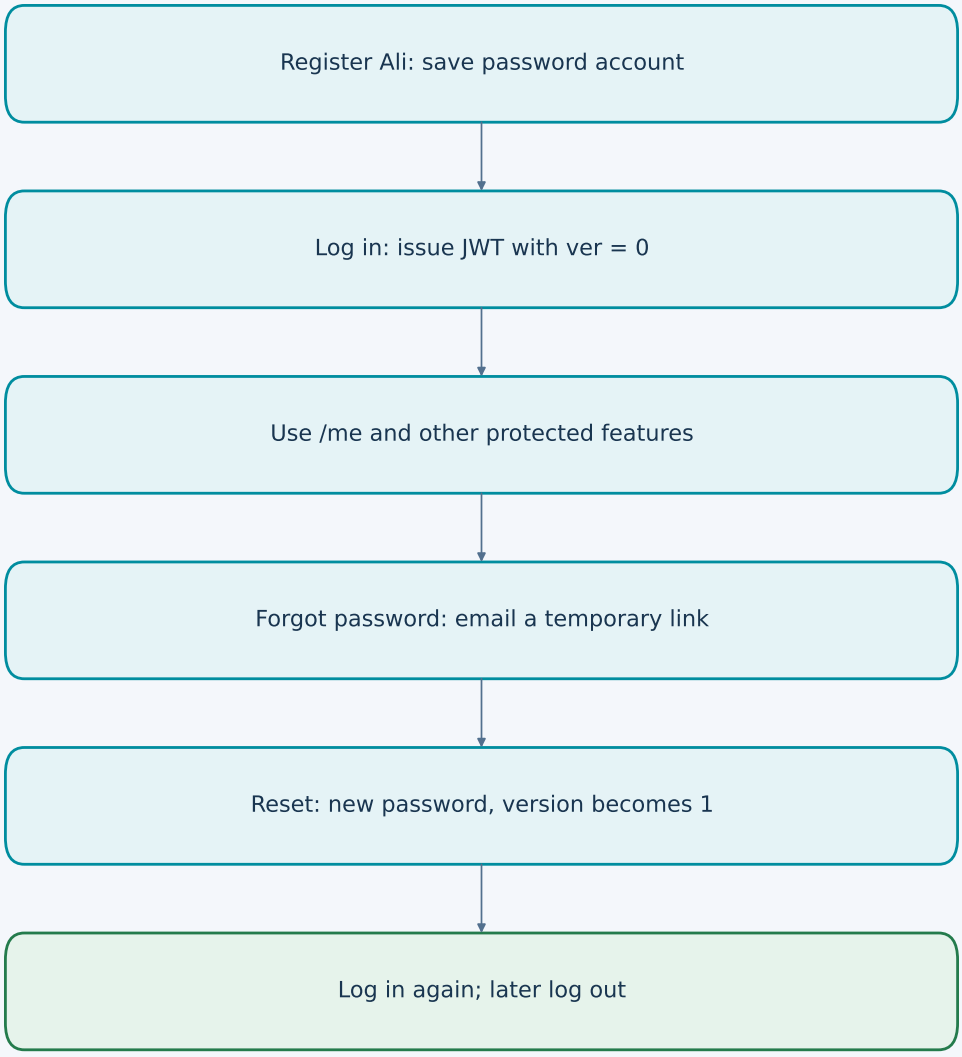
```
Vault Agent injects DATABASE_URL, JWT_SECRET_KEY, SMTP_PASSWORD.
Compose supplies Google client ID, reset URL and mail sender.
```

Missing database/JWT settings prevent startup; unavailable Google/reset configuration gives 503 on relevant requests. SMTP configuration failures are logged. Frontend/backend Google client IDs must match.

22. Follow the complete user lifecycle

Password account / Ali's account and sessions

Creating an account, signing in and resetting a password are separate operations. Account data survives backend restarts because it lives in PostgreSQL.



Keep the concepts separate

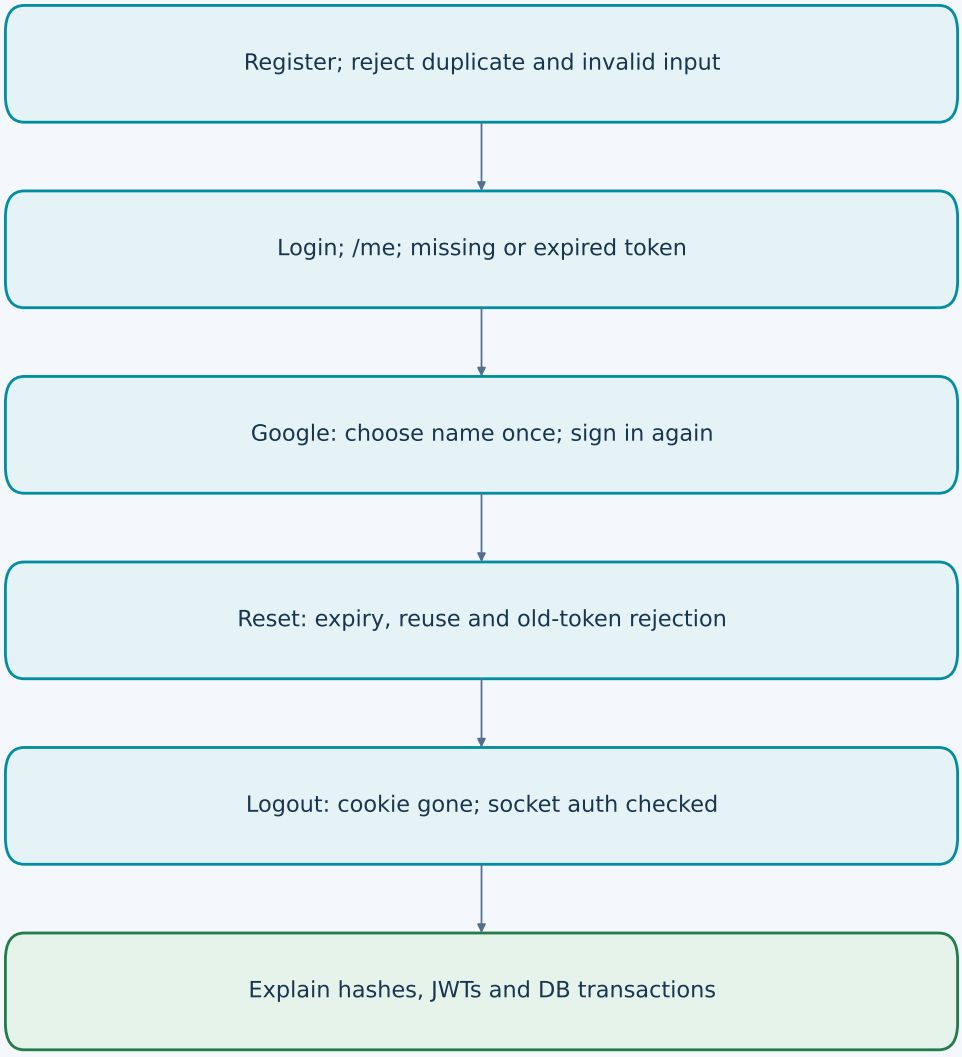
```
Google identity proof -> local application JWT -> protected request
Reset token -> password change -> old sessions fail their next check
```

There is no refresh-token endpoint, automatic Google account linking or email-ownership verification for password signup in these modules. Existing Google usernames stay unchanged.

23. What to demonstrate and explain

Evaluation checklist / current behavior

Use test accounts to demonstrate real behavior. The diagrams document the current source; creating this PDF did not run the API, Google sign-in or real email delivery.



Know the practical limits

```
No login/IP limiter in this folder; reset cooldown is per account.
No durable email retries; logout does not revoke a copied JWT.
```

Database constraints protect account uniqueness. Conditional reset consumption handles competing requests. HTTPS is deployment's job. CORS is not authentication; permissions remain feature-specific.